МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МИРЭА — Российский технологический университет»
(РТУ МИРЭА)

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

Дисциплина: «Технологии индустриального программирования»

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ № 2
ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ ПРОЕКТА (SRS) И ВИЗУАЛЬНОЕ МОДЕЛИРОВАНИЕ UML

Тема проекта: «MySearchEngine — локальная высокоскоростная поисковая система с нечетким поиском»

Выполнил(а):
Студент: Чупров Д. / Счётчиков О.
Группа: ЭФБО-03-25

Проверил:
Преподаватель:
Афонин Сергей Александрович
Учебный год: 2026–2027 гг.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ **3**

1.1. Цель документа и проекта 3

1.2. Область действия (Scope) 3

1.3. Термины, определения и сокращения 4

2. ОБЩЕЕ ОПИСАНИЕ СИСТЕМЫ **6**

2.1. Контекст и архитектурное окружение продукта 6

2.2. Основные функции продукта 6

2.3. Характеристики и квалификация пользователей 6

2.4. Предположения и зависимости 6

3. СПЕЦИФИКАЦИЯ ТРЕБОВАНИЙ **8**

3.1. Методология выявления и анализа требований 8

3.2. Бизнес-требования (Business Requirements) 8

3.3. Пользовательские требования (User Requirements) 9

3.4. Функциональные требования (Functional Requirements) 11

3.5. Нефункциональные требования (Non-Functional Requirements) 12

4. АРХИТЕКТУРНЫЕ UML-ДИАГРАММЫ **15**

4.1. Диаграмма вариантов использования (Use Case Diagram) 15

4.2. Диаграмма классов (Class Diagram) 17

4.3. Диаграмма последовательности (Sequence Diagram) 18

5. ЗАКЛЮЧЕНИЕ **19**

1. Введение

1.1. Цель документа и проекта

Настоящий документ представляет собой Спецификацию требований к программному обеспечению (Software Requirements Specification, SRS) для проектирования и разработки локальной высокоскоростной поисковой системы «MySearchEngine». Документ разработан в строгом соответствии с требованиями практического задания по дисциплине «Технологии индустриального программирования» и предназначен для фиксации полного набора архитектурных, функциональных и эксплуатационных спецификаций, обеспечивающих однозначное понимание рамок и результатов проекта между разработчиками, тестировщиками и контролирующим преподавателем.

Главной целью программного проекта «MySearchEngine» является создание автономного, отказоустойчивого и высокопроизводительного консольного поискового движка на языке программирования C++, способного выполнять мгновенную индексацию локальных неструктурированных текстовых массивов данных, обеспечивать непрерывный диалоговый поиск в интерактивном консольном режиме (REPL) и реализовывать интеллектуальное исправление пользовательских опечаток на русском и английском языках без привлечения внешних СУБД или готовых коммерческих движков.

1.2. Область действия (Scope)

Программная система «MySearchEngine» ориентирована на локальное использование на рабочих станциях пользователей. В границы проектируемой системы входят следующие ключевые возможности и компоненты:

— **Модуль обхода файловой структуры и индексации:** рекурсивное обнаружение и чтение всех текстовых файлов с расширением .txt в заданной директории по явной команде пользователя rescan с поддержкой добавления, изменения и удаления документов;

— **Модуль лингвистической предобработки:** сегментация текста на предложения, токенизация на отдельные слова, удаление пунктуационных знаков, спецсимволов и цифр, нормализация регистра букв для кириллицы и латиницы в стандарте UTF-8;

— **Модуль инвертированного индексирования:** построение и актуализация компактной хэш-таблицы в оперативной памяти (In-Memory Inverted Index), связывающей уникальные термы с идентификаторами документов (DocID) и номерами содержащих их предложений (SentenceIdx);

— **Модуль интеллектуального исправления опечаток:** вычисление расстояния Дамерау-Левенштейна между ошибочно введенным термом и словарем индекса методом динамического программирования с оптимизацией по памяти;

— **Модуль ранжирования и сниппетирования:** вычисление частоты совпадений термов (Term Frequency), упорядочивание результатов по убыванию релевантности и формирование контекстных превью (1–3 предложения с найденными словами);

— **Пользовательский интерфейс:** интерактивная консольная среда (REPL-цикл), поддерживающая многократные запросы, команды управления каталогом и запуска индексации (rescan), а также корректное завершение работы по управляющей команде :q.

В рамки текущей версии системы намеренно не входят: обработка проприетарных бинарных форматов (.pdf, .docx), графический интерфейс (GUI), сетевое клиент-серверное взаимодействие, распределенная кластеризация данных и применение внешних реляционных или нереляционных СУБД (PostgreSQL, Elasticsearch, Lucene и др.).

1.3. Термины, определения и сокращения

В таблице 1 представлены ключевые термины, определения, алгоритмические понятия и аббревиатуры, используемые в настоящей спецификации и исходном коде проекта.

Таблица 1 — Глоссарий терминов и определений предметной области

Термин / Аббревиатура	Определение и семантическое значение в системе
Поисковый запрос (Query)	Строка текста (одно или несколько слов), вводимая пользователем через интерактивный консольный интерфейс для обнаружения содержащих эти слова документов.
Релевантный документ	Документ корпуса, содержимое которого соответствует поисковому запросу. Степень релевантности рассчитывается математически на основе частоты встречаемости слов запроса.
Токен (Token)	Минимальная смысловая единица (последовательность символов, как правило, отдельное слово), полученная в результате лексического анализа текста.
Токенизация / Парсинг	Процесс разделения исходного неструктурированного текста на предложения и отдельные элементарные токены (слова).
Лингвистическая предобработка (Нормализация)	Приведение токенов к стандартной форме: очистка от пунктуации, спецсимволов и цифр, а также приведение к нижнему регистру (Case Insensitivity) с поддержкой кириллицы и латиницы.
Инвертированный индекс (Inverted Index)	Ключевая структура данных поискового движка. Представляет собой ассоциативный массив (хэш-таблицу), где ключом выступает уникальное нормализованное слово, а значением — список вхождений (постинг-лист) с идентификаторами документов и координатами предложений.
Сниппет (Snippet)	Контекстный фрагмент текста документа (в системе — 1–3 предложения), содержащий искомое слово. Позволяет пользователю мгновенно оценить контекст файла без его открытия.
Ранжирование (Ranking)	Алгоритмический процесс сортировки документов по степени соответствия запросу. Документы с большим количеством совпадений получают высший ранг и выводятся первыми.
Расстояние Дамерау-Левенштейна	Метрика редактирования строк, определяющая минимальное число операций (вставка, удаление, замена символа, перестановка двух соседних символов — транспозиция), необходимых для преобразования одной строки в другую.

Термин / Аббревиатура	Определение и семантическое значение в системе
REPL (Read-Eval-Print Loop)	Консольный интерактивный цикл «чтение-вычисление-вывод», в котором система непрерывно принимает запросы, производит поиск, выводит результаты и ожидает следующий ввод без перезапуска программы.
DocID (Document Identifier)	Уникальный целочисленный идентификатор, присваиваемый документу при первичном сканировании каталога для компактной адресации в инвертированном индексе.
SentenceIdx	Порядковый индекс предложения в документе, используемый для точной привязки терма к контексту и быстрого извлечения сниппета.
SOLID	Пять фундаментальных принципов объектно-ориентированного программирования и проектирования (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion), обеспечивающих расширяемость и сопровождаемость кодовой базы.
UTF-8	Стандарт кодирования Unicode переменной длины, используемый для корректного представления и обработки русскоязычного и англоязычного текста.
SRS	Software Requirements Specification — Спецификация требований к программному обеспечению.

2. Общее описание системы

2.1. Контекст и архитектурное окружение продукта

Программный продукт «MySearchEngine» спроектирован как автономное высокопроизводительное консольное приложение (CLI), функционирующее непосредственно в среде операционной системы (macOS, Linux, Windows) и не зависящее от сторонних серверов приложений или СУБД. Взаимодействие с пользователем организовано через стандартные потоки ввода-вывода (std::cin, std::cout) терминала, что обеспечивает минимальные накладные расходы и возможность работы на серверах без графической оболочки.

Архитектура системы строится на принципах слабой связанности (Low Coupling) и высокой зацепленности (High Cohesion). Исходный код разделен на заголовочные интерфейсы в директории include/ (engine.hpp, spellchecker.hpp) и реализацию в src/ (main.cpp). Все структуры данных размещаются в оперативной памяти (In-Memory Processing), что гарантирует максимальное быстродействие при обработке запросов.

2.2. Основные функции продукта

Система предоставляет пользователю следующий набор базовых макрофункций:

— **Индексация файлового хранилища по команде:** сканирование, фильтрация и структурирование всех .txt файлов в указанном пользователем каталоге по явной команде rescan; поддержка полной переиндексации при добавлении, изменении и удалении документов с выводом суммарного отчета;

— **Точный полнотекстовый поиск:** мгновенное сопоставление нормализованных слов запроса со словарем инвертированного индекса за время O(1);

— **Нечеткий поиск (Fuzzy Search):** интеллектуальный подбор ближайшего словарного аналога при наличии опечаток с последующим автоматическим выполнением поиска по исправленному терму;

— **Релевантная выдача:** сортировка найденных документов по числу совпадений с формированием информативных блоков, включающих абсолютный путь к файлу и релевантные цитаты (сниппеты);

— **Управление диалоговой сессией:** интерактивное взаимодействие в цикле REPL с возможностью повторных запросов без повторной ресурсоемкой индексации и быстрый выход по команде :q.

2.3. Характеристики и квалификация пользователей

Целевыми пользователями системы являются студенты, инженеры-разработчики, системные администраторы и исследователи, которым требуется оперативный поиск информации по локальным архивам текстовой документации, конспектов, технических отчетов или программных логов. От пользователя требуется минимальный уровень компьютерной грамотности: умение запускать исполняемые файлы в терминале, указывать относительные и абсолютные пути к директориям файловой системы и вводить текстовые команды.

2.4. Предположения и зависимости

Успешное функционирование поискового движка базируется на следующих системных предположениях и зависимостях:

— **Аппаратная среда:** объем оперативной памяти рабочей станции должен быть достаточен для удержания инвертированного индекса и текста предложений (для текстовых корпусов объемом до 500 МБ требуется не менее 1 ГБ свободной RAM);

— **Программная среда:** наличие компилятора с поддержкой стандарта C++17 или новее (GCC 9+, Clang 10+, MSVC 2019+), поддержка стандартной библиотеки std::filesystem и локали UTF-8;

— **Кодировка данных:** все исходные индексируемые текстовые файлы должны быть сохранены в кодировке UTF-8 или ASCII;

— **Права доступа:** пользователь, запускающий приложение, должен обладать правами на чтение целевой директории и всех содержащихся в ней поддиректорий и файлов.

3. Спецификация требований

3.1. Методология выявления и анализа требований

В рамках выполнения практического задания сбор и структурирование требований производились на основе признанной в промышленной программной инженерии методологии Карла Вигерса (Karl Wiegers). Согласно данной иерархической модели, требования декомпозируются на четыре последовательных и взаимоувязанных уровня:

— **1. Бизнес-требования (Business Requirements):** определяют стратегические цели создания продукта, критерии рыночного или учебного успеха и основные ресурсные ограничения;

— **2. Пользовательские требования (User Requirements):** описывают цели и задачи, которые конечный пользователь может решать с помощью программной системы (в форме сценариев использования и прецедентов);

— **3. Функциональные требования (Functional Requirements):** детализируют конкретное поведение и логику программных модулей системы при выполнении пользовательских задач;

— **4. Нефункциональные требования (Non-Functional Requirements):** задают качественные атрибуты системы: архитектурные шаблоны, быстродействие, алгоритмическую сложность, надежность и поддержку стандартов локализации.

Дополнительно при формулировании требований применялся анализ предметной области (Domain Analysis) теории информационного поиска (Information Retrieval) и методы проектирования прецедентов (Use Case Analysis), позволившие точно разграничить зоны ответственности каждого программного компонента.

3.2. Бизнес-требования (Business Requirements)

Бизнес-требования отражают базовую концепцию проекта и зафиксированы в таблице 2.

Таблица 2 — Спецификация бизнес-требований проекта

Идентификатор	Категория	Формулировка требования	Критерий приемки
BR-1	Бизнес-цель	Ускорение поиска информации по текстовым файлам, с исправлением опечаток.	Существенное сокращение времени пользователя на поиск информации по текстовым архивам; моментальная локальная выдача релевантных совпадений и автоматическая коррекция опечаток в запросах на русском и английском языках.

Идентификатор	Категория	Формулировка требования	Критерий приемки
BR-2	Критерии успеха	Получение рабочего консольного приложения, способного быстро обрабатывать текстовые файлы и находить релевантную информацию, экономя время пользователя на ручной поиск.	Время ответа на поисковый запрос по массиву проиндексированных документов не превышает 50 миллисекунд.
BR-3	Бизнес-ограничения	Разработка ведется в рамках учебной дисциплины силами студентов. Требуется самостоятельная реализация ключевых алгоритмов (инвертированный индекс, расчет расстояния редактирования) на языке C++ без использования сторонних СУБД или готовых полнотекстовых движков.	Отсутствие зависимостей от сторонних СУБД и сторонних поисковых библиотек. Чистая реализация на C++17/STL.

3.3. Пользовательские требования (User Requirements)

Пользовательские требования специфицируют функциональные ожидания пользователя от взаимодействия с системой через диалоговый терминал. Сценарий работы пользователя строится следующим образом: пользователь запускает программу, указывает путь к рабочей директории с файлами. Индексация не выполняется автоматически при старте — процесс индексирования запускается строго по явной команде пользователя rescan. В ходе дальнейшей работы при модификации содержимого директории (добавление новых файлов, изменение существующих или удаление файлов) пользователь повторно вводит команду rescan, запуская полную переиндексацию без необходимости перезапуска приложения.

В рамках модели взаимодействия формализованы пять ключевых путей пользователя (User Journeys):

— **1-й путь пользователя (Поиск):** интерактивный ввод поисковых запросов в терминале (REPL-цикл) без перезапуска программы;

— **2-й путь пользователя (Автоисправление):** интеллектуальный поиск с автоисправлением опечаток на русском и английском языках;

— **3-й путь пользователя (Выдача):** получение ранжированных результатов по релевантности с абсолютными путями к файлам и превью-сниппетами;

— **4-й путь пользователя (Индексация):** указание пути к директории и запуск индексации/переиндексации по команде rescan с обязательной поддержкой добавления, изменения и удаления документов;

— **5-й путь пользователя (Завершение):** корректное освобождение ресурсов и выход из приложения по управляющей команде :q.

Спецификация требований пользователей с указанием приоритетов по методологии MoSCoW и сценариев использования представлена в таблице 3.

Таблица 3 — Спецификация пользовательских требований

ID	Требование пользователя	Приоритет (MoSCoW)	Сценарий использования
UR-1	Пользователь должен иметь возможность непрерывно вводить поисковые запросы через интерактивную консоль (REPL) без необходимости перезапуска программы.	Must Have	После завершения поиска система выводит приглашение ко вводу следующего запроса.
UR-2	Если пользователь допускает опечатку в запросе, он должен всё равно получать корректные результаты поиска по наиболее близкому исправленному слову.	Must Have	При вводе ошибочного терма система уведомляет: «Возможно, вы имели в виду: ...» и отображает результаты.
UR-3	Пользователь должен получать список результатов, отсортированный по релевантности (по количеству совпадений).	Must Have	Документы с максимальной плотностью совпадений выводятся первыми в списке выдачи.
UR-4	Пользователь должен иметь возможность указать путь к директории и вручную запустить индексацию по команде rescan. Система обязана поддерживать полную переиндексацию: удаление, добавление и изменение документов.	Must Have	Пользователь заходит в программу, указывает директорию и вводит команду rescan. Система актуализирует инвертированный индекс с учетом всех изменений.
UR-5	Для понимания контекста найденного совпадения пользователь должен видеть в выдаче абсолютный путь к файлу и превью-сниппеты (до 3 предложений из текста документа).	Should Have	Для каждого документа отображаются путь и до 3 цитат с подсветкой или указанием контекста.

ID	Требование пользователя	Приоритет (MoSCoW)	Сценарий использования
UR-6	Пользователь должен иметь возможность корректно завершить работу приложения вводом управляющей команды (например, :q или exit).	Must Have	При вводе :q или exit ресурсы освобождаются, и процесс штатно завершается с кодом 0.

3.4. Функциональные требования (Functional Requirements)

Функциональные требования определяют поведение программных компонентов движка при обработке данных (таблица 4).

Таблица 4 — Спецификация функциональных требований

ID	Функция системы	Детальное описание алгоритмического поведения	Входные и выходные данные
FR-1	Сканирование файловой структуры по команде rescan	Система осуществляет рекурсивный обход выбранного пользователем каталога с помощью std::filesystem::recursive_directory_iterator только по явной команде пользователя rescan, отбирая только регулярные файлы с расширением .txt и определяя добавленные, измененные и удаленные файлы.	Вход: путь к каталогу, команда rescan. Выход: вектор путей к актуальным .txt файлам.
FR-2	Лингвистическая предобработка текста	Система разбивает сырой текст документов на предложения, очищает слова от знаков пунктуации, спецсимволов и цифр, выполняет перевод всех символов в нижний регистр (Case Folding) для кириллицы и латиницы.	Вход: сырой текст файла. Выход: нормализованные токены и структурированные предложения.

ID	Функция системы	Детальное описание алгоритмического поведения	Входные и выходные данные
FR-3	Построение и перестроение инвертированного индекса	Система строит инвертированный индекс в оперативной памяти (Term -> PostingList). При выполнении команды rescan система осуществляет полную переиндексацию: удаляет устаревшие записи удаленных файлов, индексирует новые и обновляет измененные документы.	Вход: поток токенов документов. Выход: актуализированная структура InvertedIndex в RAM.
FR-4	Поддержка REPL-цикла и управляющих команд	Система поддерживает интерактивный цикл «чтение-вычисление-вывод», ожидая строковый ввод от пользователя, парсит введенную строку на команды (указание директории, команда rescan, выход :q) или поисковые термы.	Вход: ввод из std::cin (команды или термы). Выход: передача управления модулю индексации, поиска или выхода.
FR-5	Автоисправление опечаток	При отсутствии терма в словаре индекса система активирует расчет расстояния Дамерау-Левенштейна между запросом и всеми словами словаря. Выбирается слово с минимальным расстоянием, не превышающим порог (<= 2).	Вход: отсутствующий терм, словарь. Выход: исправленный терм-кандидат.
FR-6	Ранжирование и формирование выдачи	Система производит агрегацию вхождений найденных документов, сортирует массив совпадений по убыванию Term Frequency и извлекает до 3 предложений для каждого релевантного документа.	Вход: PostingList совпадений. Выход: консольный отчет с путями и сниппетами.

3.5. Нефункциональные требования (Non-Functional Requirements)

Нефункциональные требования формулируют критерии качества программной реализации, ограничения производительности, надежности и архитектурные паттерны (таблица 5).

Таблица 5 — Спецификация нефункциональных требований

ID	Категория	Спецификация требования	Механизм технической реализации
NFR-1	Архитектура и качество кода (SOLID)	Проектирование и реализация системы должны опираться на принципы объектно-ориентированного дизайна SOLID. Каждый класс имеет строго одну зону ответственности (SRP), интерфейсы изолированы.	Разделение на независимые классы: SearchEngine, InvertedIndex, SpellChecker, DocumentStore, TextProcessor, REPL.
NFR-2	Производительность индекса и переиндексации	Инвертированный индекс обеспечивает поиск точного слова за время O(1) в среднем. Время ответа <= 50 мс. Производительность: - [ ] Полная переиндексация (удаление, добавление, изменение): модуль индексации по команде rescan гарантирует высокую скорость актуализации индекса без перезапуска приложения.	Хэш-таблица std::unordered_map<std::wstring, PostingList> с эффективной хэш-функцией; быстрое обновление списков постингов и реестра DocumentStore.
NFR-3	Алгоритмическая оптимизация памяти	Алгоритм нечеткого сопоставления должен реализовывать метрику Дамерау-Левенштейна методом динамического программирования с экономией памяти O(min(M, N)).	Использование двух строк рабочей матрицы размером min(M, N) вместо хранения всей двумерной таблицы M x N.
NFR-4	Надежность и отказоустойчивость	Система должна корректно игнорировать нетекстовые файлы, поврежденные кодировки, ошибки ввода несуществующих директорий и пустые поисковые строки без аварийного падения (Crash-free).	Блоки try-catch, валидация путей через std::filesystem::exists, проверка регулярности файлов, информативные сообщения об ошибках.

ID	Категория	Спецификация требования	Механизм технической реализации
NFR-5	Локализация и кодировки	Полная и корректная обработка кириллических и латинских символов во всех вычислительных модулях (чтение, токенизация, расчет расстояния редактирования).	Применение широких строк std::wstring, перевод через std::wstring_convert и локаль ru_RU.UTF-8 (std::towlower, std::iswpunct).

4. Архитектурные UML-диаграммы

В соответствии с заданием на практическое занятие, в данном разделе представлены три ключевые диаграммы унифицированного языка моделирования (UML 2.5), описывающие систему в трех различных проекциях: функциональной (Use Case Diagram), структурной (Class Diagram) и поведенческой во времени (Sequence Diagram).

4.1. Диаграмма вариантов использования (Use Case Diagram)

Диаграмма вариантов использования визуализирует границы проектируемой системы MySearchEngine, взаимодействующих с ней акторов и реализуемые прецеденты (рисунок 1). В системе выделены два актора:

— **Пользователь (первичный актор):** студент, аналитик или разработчик, указывающий путь к директории, запускающий индексацию/переиндексацию по явной команде rescan (4-й путь пользователя), выполняющий поисковые запросы через интерактивную консоль и завершающий сеанс работы;

— **Файловая система (вторичный актор / внешняя среда):** подсистема операционной системы, предоставляющая доступ к древовидной структуре файлов и текстовым потокам .txt.

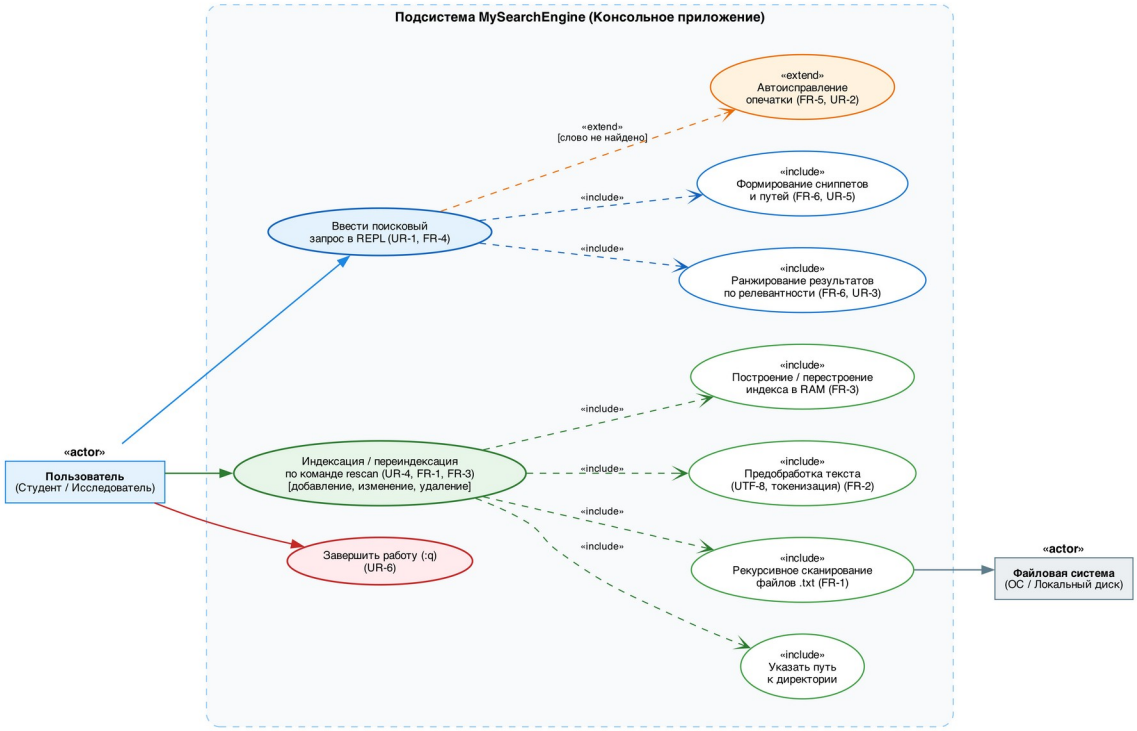


Рисунок 1 — Диаграмма вариантов использования подсистемы MySearchEngine

Диаграмма демонстрирует декомпозицию основных прецедентов с использованием стандартных отношений UML:

— **Отношение включения «include»:** прецедент «Индексация / переиндексация по команде rescan» безусловно включает в себя шаги «Указать путь к директории», «Рекурсивное сканирование файлов .txt», «Предобработка текста» и «Построение / перестроение индекса в RAM» (с поддержкой удаления, добавления и изменения документов). Аналогично, прецедент «Ввод поискового запроса» включает «Ранжирование результатов» и «Формирование контекстных сниппетов»;

— **Отношение расширения «extend»:** прецедент «Автоисправление опечатки» активируется только при выполнении условия расширения: запрашиваемый терм отсутствует в инвертированном индексе, что инициирует алгоритмический подбор ближайшего словарного аналога.

4.2. Диаграмма классов (Class Diagram)

Диаграмма классов представляет статическую объектно-ориентированную структуру системы MySearchEngine, демонстрируя основные сущности, их атрибуты, методы, области видимости (+ public, - private) и межклассовые отношения (рисунок 2).

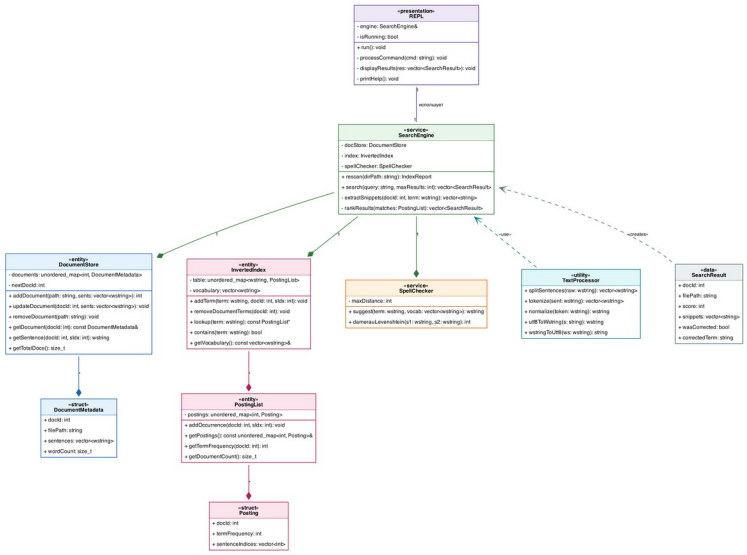


Рисунок 2 — Диаграмма классов программной архитектуры поисковой системы

Архитектура системы включает следующие ключевые классы и структуры данных:

— **REPL (Presentation Layer):** управляет циклом взаимодействия с пользователем в терминале, считывает строки запросов и команд (путь, команда rescan, выход), вызывает методы движка SearchEngine и выводит форматированные отчеты выдачи;

— **SearchEngine (Facade & Coordinator):** центральный фасад системы, координирующий работу хранилища документов, инвертированного индекса и модуля исправления опечаток. Реализует метод rescan() для полной переиндексации корпуса;

— **DocumentStore:** хранилище нормализованных текстовых документов и их метаданных (DocumentMetadata). Отвечает за сопоставление DocID с физическим путем к файлу, выдачу предложений и операции добавления, модификации и удаления файлов при переиндексации;

— **InvertedIndex:** высокопроизводительный ассоциативный индекс на базе std::unordered_map<std::wstring, PostingList>. Обеспечивает мгновенный доступ к постинг-листам по строковому ключу терма и удаление устаревших термов при переиндексации;

— **PostingList и Posting:** структуры, содержащие списки идентификаторов документов (docId), частоту встречаемости терма (termFrequency) и список порядковых номеров предложений (sentenceIndices);

— **SpellChecker:** сервисный класс, реализующий оптимизированный алгоритм расчета расстояния Дамерау-Левенштейна и подбор наиболее правдоподобных исправлений из словаря;

— **TextProcessor:** статический утилитный класс (Utility Class), инкапсулирующий строковые алгоритмы разбиения на предложения, токенизации, фильтрации знаков препинания и конвертации между std::string (UTF-8) и std::wstring;

— **SearchResult:** структура данных передачи результата (DTO), содержащая путь к документу, вычисленный ранг релевантности, сниппеты и флаг произведенного автоисправления.

4.3. Диаграмма последовательности (Sequence Diagram)

Диаграмма последовательности детализирует динамику выполнения наиболее критического и сложного пользовательского сценария: «Обработка поискового запроса с нечетким поиском (исправлением опечатки), ранжированием и генерацией сниппетов» (рисунок 3).

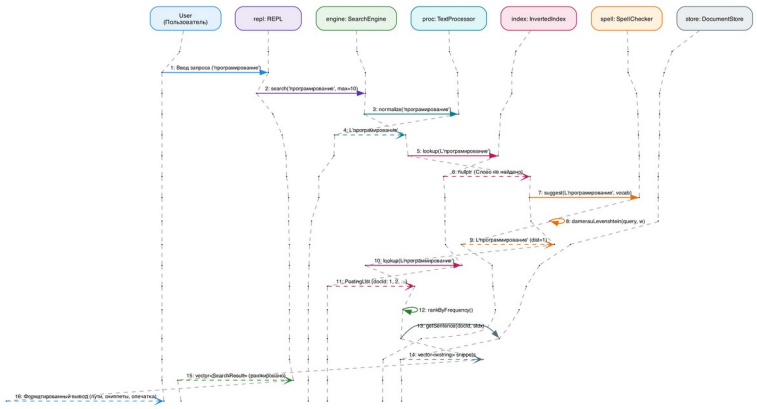


Рисунок 3 — Диаграмма последовательности обработки поискового запроса с автоисправлением

Хронологическая последовательность обмена сообщениями между объектами включает 16 последовательных шагов:

— **Шаги 1–2:** пользователь вводит в интерактивном терминале поисковый запрос со случайной опечаткой («програмирование»). Класс REPL считывает строку и инициирует вызов engine.search("програмирование", 10);

— **Шаги 3–4:** SearchEngine передает сырую строку в TextProcessor::normalize, где происходит приведение к нижнему регистру и конвертация в UTF-8/wstring;

— **Шаги 5–6:** SearchEngine выполняет запрос index.lookup(L"програмирование"). Поскольку слово содержит опечатку, InvertedIndex возвращает nullptr (промах в индексе);

— **Шаги 7–9:** SearchEngine обнаруживает промах и делегирует поиск ошибки классу SpellChecker, передавая ошибочный терм и словарь индекса index.getVocabulary(). SpellChecker циклически рассчитывает расстояние Дамерау-Левенштейна, находит ближайшего кандидата («программирование» с расстоянием 1) и возвращает его движку;

— **Шаги 10–11:** SearchEngine повторно запрашивает инвертированный индекс с исправленным словом: index.lookup(L"программирование"). Индекс возвращает соответствующий PostingList с коллекцией документов и позициями;

— **Шаг 12:** SearchEngine производит ранжирование документов по количеству найденных совпадений (Term Frequency Ranking);

— **Шаги 13–14:** для документов, вошедших в итоговый топ выдачи, SearchEngine запрашивает у DocumentStore исходные предложения по сохраненным SentenceIdx и формирует контекстные превью (сниппеты);

— **Шаги 15–16:** SearchEngine упаковывает сформированные SearchResult (включая уведомление о подстановке исправленного слова) и возвращает в REPL, который осуществляет форматированный вывод информации на терминал пользователя.

5. Заключение

В ходе выполнения Практического занятия № 2 была успешно разработана исчерпывающая спецификация требований к программному обеспечению (SRS) для поисковой системы «MySearchEngine» на основе требований, собранных на предшествующем этапе проектирования.

В документе в строгом соответствии с академическими стандартами и методологией Карла Вигерса структурированы бизнес-требования, требования пользователей, функциональные и нефункциональные характеристики системы. Создан полный комплект архитектурных UML-диаграмм (Use Case Diagram, Class Diagram, Sequence Diagram), детализирующих функциональные прецеденты, внутреннюю статическую структуру объектно-ориентированных классов C++ и динамическое поведение компонентов при обработке поисковых запросов с исправлением опечаток.

Подготовленная документация служит надежной проектной основой для последующей кодогенерации, модульного тестирования и внедрения системы в рамках дисциплины «Технологии индустриального программирования».